

Progetto per il Corso Introduzione alla Programmazione per il Web a.a. 2025-2026

Istruzioni da leggere attentamente

Modalità di consegna:

La consegna del progetto deve essere effettuata su Moodle nella sezione Progetto nella cartella creata per l'appello a cui vi iscrivetevi (es. AppelloLuglio). Ad ogni appello, verrà accettata una sola consegna per team. Quindi, in caso di consegne da parte di più studenti appartenenti allo stesso team, verrà valutata unicamente l'ultima consegna ricevuta alla scadenza prevista.

La consegna consiste di cinque file:

1. Un file pdf contenente la relazione. La relazione del progetto deve essere redatta sull'apposito template che potete scaricare da Moodle nella sezione Progetto. La lunghezza della relazione è di massimo 7 pagine tutto incluso. Il nome del file deve essere *Relazione_teamid*.
2. Una cartella zippata contenente il codice della web app principale. Il nome della cartella deve essere *webapp_teamid*.
3. Una cartella zippata contenente il codice del servizio REST-like. Il nome della cartella deve essere *REST_service_teamid*.
4. Il .jar della web app
5. Il .jar del servizio REST-like

Teamid è l'identificativo del gruppo riportato nell'apposito file su Moodle nella sezione Progetto. Moodle supporta una consegna di file di dimensione massima pari a 100MB.

Per la consegna dei .jar: dopo il deployment, recuperare il .jar dalla cartella target e copiarlo altrove rinominandolo (togliere 0.0.1 SNAPSHOT). Consegnare solo questo .jar.

Per la consegna dei file zippati: prima di fare lo zip, eliminate le cartelle target e .idea per limitare le dimensioni della consegna

Consiglio per limitare la dimensione dei file: usare immagini a bassa risoluzione.

Data di consegna:

Per venire incontro alle esigenze degli studenti, la data della consegna per l'appello di Giugno è fissata al 16/06/2026 alle 13:00. Per gli altri appelli, la consegna è fissata a 3 giorni prima della data del quiz.

Contributo membri del team:

Ciascun membro del team deve contribuire sia lato back end sia lato front end, altrimenti sarà applicata una penalità sul punteggio finale del progetto.

Non è richiesto che i membri del gruppo sostengano l'esame nello stesso appello, ma il progetto va consegnato al primo appello a cui un membro del gruppo partecipa.

Riconsegna:

Se il progetto risulta non sufficiente, potete riconsegnarlo quante volte volete fino al raggiungimento della sufficienza. Se il progetto risulta sufficiente, potete riconsegnarlo ancora una volta nell'anno, all'appello che preferite. In ogni caso, dovete nominare i file consegnati aggiungendo "v2" e

consegnare un file supplementare in formato pdf chiamato *Modifiche* riportante la lista delle modifiche apportate al progetto rispetto alla sua versione originale.

Tecnologie:

Il progetto deve essere sviluppato utilizzando *esclusivamente* le tecnologie viste durante il corso, ovvero HTML, CSS, Bootstrap, tutta la parte per l'implementazione in Spring del back-end, JavaScript, AJAX, etc.

Attenzione! Usate le versioni di IntelliJ, JDK e Spring Boot e OpenFeign viste a lezione e riportate nelle slide.

L'utilizzo di tecnologie diverse porterà ad una consistente penalità. *Nota: tecnologie non significa funzioni o simili.*

Un accenno particolare alla tecnologia del DB:

- Si usi H2 come DBMS.
- Si consegnino tutti i file riportanti le istruzioni SQL per creare / popolare le tabelle usate dal/dai DB. Tali file devono avere estensione .sql

Il progetto deve usare almeno 3 elementi di Bootstrap scelti a piacere.

Valutazione:

Il progetto vale il 50% del voto finale dell'esame e verrà valutato solo se gira. La mancata consegna di alcune parti del progetto non implica automaticamente l'insufficienza.

I criteri di valutazione sono i seguenti:

- 1) Architettura e design:
 - Organizzazione in livelli logici chiari
 - Uso corretto dei principi di progettazione
- 2) Funzionalità:
 - Tutte le funzionalità sono presenti e implementano la logica richiesta
- 3) Qualità del codice:
 - Codice leggibile e chiaro
- 4) Tecnologie integrate:
 - Spring Security configurato correttamente
 - OpenFeign configurato correttamente
 - Frontend dinamico implementato correttamente
 - Database H2 configurato e funzionante
- 5) Completezza, correttezza e chiarezza della relazione

In caso di progetti identici o molto simili tra due o più team, il voto del progetto verrà diviso tra tutti i componenti del progetto. *Esempio: per due progetti identici e quattro membri totali, il voto finale sarà: voto/4.*

Ricordate che ChatGPT o affini non fanno parte del team e quindi non partecipano al progetto ☺

Vostro cugino gestisce un centro fitness e benessere e desidera offrire ai propri clienti la possibilità di consultare e gestire i propri programmi di allenamento tramite la web. Sapendo delle vostre competenze nello sviluppo software, vostro cugino vi chiede aiuto per l'implementazione di una web app che supporti la gestione di queste informazioni in modo semplice ed efficace.

L'applicazione deve verificare i requisiti strutturali propri del design pattern MVC. La web app usa alcuni servizi interni e un servizio REST-like.

La web app consiste di una parte pubblica e di una parte privata, quest'ultima accessibile attraverso un'autenticazione con login e password da quattro tipologie di utenti descritte in seguito. Tutte le pagine della web app sono disponibili in italiano. Per ciascuna delle pagine il titolo della tab del browser in cui viene visualizzata la pagina web è il nome della web app (da voi scelto) che sarà corredato con il logo (da voi prodotto).

La parte pubblica ha le seguenti pagine:

- 1) Una *home page* (chiamata *index*) che deve riportare:
 - a. il nome del centro e il suo logo;
 - b. un'immagine;
 - c. una descrizione testuale della web app che spieghi a cosa serve, per chi è e perché gli utenti dovrebbero usarla;
 - d. i dati fittizi relativi al centro: indirizzo della sede legale, mail di contatto, partita iva;
 - e. tre pulsanti di call to action per spingere l'utente a iniziare l'interazione:
 - i. *Registrati* che porta alla pagina *sign up* dove un nuovo utente può iscriversi;
 - ii. *Inizia allenamento* che permette ad un utente registrato di accedere tramite la pagina di *login* alle funzionalità della web app a cui ha accesso;
 - iii. *Contattaci* che permette ad un utente di contattare il centro tramite una form;
- 2) Una pagina *sign up* che permette ad un nuovo utente di iscriversi. La pagina contiene una form avente i seguenti campi (tutti obbligatori):
 - a. nome;
 - b. cognome;
 - c. data di nascita nel formato GG/MM/AAAA;
 - d. indirizzo email;
 - e. la scelta di uno username;
 - f. la scelta di una password che andrà inserita due volte per conferma. La password deve essere lunga 8 caratteri e deve contenere la stringa "id_xx" dove xx è il numero del vostro team;
 - g. la possibilità di selezionare il piano di allenamento preferito: *Prova*, *Basic* o *Pro*. Per ogni programma c'è una breve descrizione e il costo mensile o annuale. Il pagamento è fittizio, si suppone che la selezione del piano implichi il pagamento stesso.
 - h. un pulsante per inviare i dati;
 - i. un pulsante di reset che ripulisce tutti i campi;
- 3) Una pagina per avvisare l'utente che la sua registrazione è andata a buon fine;

- 4) Una pagina di *login* che contiene una form richiedente username e password;
- 5) Una pagina di *logout* che indica che il logout è stato effettuato correttamente;
- 6) Una pagina *contatti* che contiene una form per contattare il centro inviando un messaggio con la richiesta. La form contiene un campo per il nome, uno per l'indirizzo mail e un campo per immettere il testo del messaggio. I dati vengono inviati quando l'utente clicca su un pulsante. L'invio è fittizio e viene visualizzato un messaggio di corretto invio.

Il formato della password (nella pagina di *sign up* e nella pagina di *login*), il formato di data di nascita e il fatto che l'utente sia maggiorenne alla data di registrazione (nella pagina di *sign up*) dovranno essere validati utilizzando un opportuno script JavaScript prima di inviare i dati al server quando viene premuto il bottone *Submit*. Dopo l'avvenuta verifica dei dati della form di registrazione, viene invocato un servizio interno alla web app, chiamato *CheckUser*, che accede ad un database embedded (vedi sotto) che contiene i dati degli utenti registrati e verifica l'esistenza o meno di un utente con lo stesso username. Se esiste un utente con lo stesso username, viene presentata nuovamente la pagina *sign up* con un messaggio aggiuntivo di errore. Altrimenti, i dati del nuovo utente vengono inseriti nel database, insieme alla data di sign up e viene visualizzata la conferma della registrazione. Per inserire la data si usi `java.sql.Date.valueOf("data")` come uno dei parametri di `jdbcTemplate.update`.

L'indirizzo mail nella pagina contatti è fittizio e, per semplicità di realizzazione, non se ne richiede la verifica di correttezza del formato.

La webapp deve supportare quattro tipologie di utenti:

- `ROLE_ADMIN`
- `ROLE_USER_PROVA`
- `ROLE_USER_BASIC`
- `ROLE_USER_PRO`

Le quattro tipologie di utenti possono accedere a funzionalità diverse, come specificato nel seguito. `ROLE_USER_PROVA` resterà attivo (abilitato all'uso della web app) fino al completamento di tre allenamenti. Al termine l'utente viene disabilitato e per accedere deve ripetere il *sign up*.

Quando un utente registrato effettua l'accesso, il server verifica la correttezza delle credenziali username e password a cui è stato applicato uno degli algoritmi di hashing resi disponibili da Spring Security. Se le credenziali sono errate, gli utenti vengono riportati alla pagina di *login* e viene mostrato il messaggio di errore `"#team_id: That user is not authenticated!"`. Se invece, le credenziali sono corrette, gli utenti possono procedere con le pagine private, ovvero le Dashboard.

Gli username che userete per *testare* i diversi ruoli sono: `admin#team_id`, `basic#team_id`, `pro#team_id`, `prova#X#team_id`, dove X è l'x-esimo `ROLE_UTENTE_PROVA`. Le password per tali utenti di test devono essere: `"adm_id_xx"`, `"bsc_id_xx"`, `"prv_id_xx"`, `"pro_id_xx"`, dove xx indica il numero del vostro team.

Dashboard ROLE_ADMIN:

La pagina mostra un messaggio di benvenuto per l'utente (e.g. "Benvenuta Giovanna!") e offre le seguenti funzionalità abilitate premendo i pulsanti seguenti:

Visualizza Lista Utenti che permette di accedere a una pagina che riporta una tabella con la lista degli utenti ordinata per ruolo e la data di registrazione.

Rimuovi utenti scaduti che permette di eliminare dal database i ROLE_USER_PROVA disabilitati. Viene visualizzato il numero di utenti rimossi.

Statistiche che permette di visualizzare una pagina con il numero medio di allenamenti per programma predefinito i ROLE_UTENTE_BASIC e ROLE_UTENTE_PRO.

Dashboard ROLE_USER_PROVA:

La pagina mostra un messaggio di benvenuto per l'utente (e.g. "Benvenuta Giovanna!") e il numero di allenamenti gratuiti di cui l'utente ha usufruito ad esempio nel formato: 2/3. Inoltre, la Dashboard offre le seguenti funzionalità abilitate premendo i pulsanti seguenti:

Visualizza Profilo che permette di accedere a una pagina con il profilo dell'utente con le seguenti informazioni: nome, cognome, data di nascita, ruolo.

Cambio Password che permette di cambiare la password associata all'utente tramite form.

Allenamento che permette di visualizzare in una pagina la lista dei programmi di allenamento *predefiniti* nella web app che vengono recuperati interrogando il servizio REST-like Programmi. Cliccando su ogni programma, viene visualizzata una scheda dettagliata del programma selezionato interrogando il servizio REST-like Programmi. Inoltre, c'è un bottone *Completato* che l'utente preme al completamento del programma. Una volta premuto il bottone, viene rivisualizzata la dashboard.

Upgrade che permette di diventare ROLE_USER-BASIC o ROLE_USER_PRO. Il pagamento è fittizio, si suppone che la selezione del piano implichi il pagamento stesso. Si segnala che l'upgrade ha avuto successo.

Inserisci recensione che permette di inserire una nuova recensione della web app. L'utente inserisce una nuova recensione tramite una form. Il bottone e la form si trovano nella dashboard. L'invio della recensione, tramite il bottone Inserisci recensione, avviene tramite AJAX e la nuova recensione viene salvata nel database embedded e visualizzata immediatamente in un carosello (vedi più avanti) senza necessità di ricaricare la pagina.

Dashboard ROLE_USER_BASIC:

La pagina Dashboard mostra un messaggio di benvenuto per l'utente (e.g. "Benvenuta Giovanna!") e offre le seguenti funzionalità abilitate premendo i pulsanti seguenti:

Visualizza Profilo che permette di accedere a una pagina con il profilo dell'utente con le seguenti informazioni: nome, cognome, data di nascita, ruolo.

Cambio Password che permette di cambiare la password associata all'utente tramite form.

Allenamento che permette di visualizzare in una pagina la lista dei programmi di allenamento *predefiniti* nella web app che vengono recuperati interrogando il servizio REST-like Programmi. Cliccando su ogni programma, viene visualizzata una scheda dettagliata del programma selezionato interrogando il servizio REST-like Programmi. Inoltre, c'è un bottone *Completato* che l'utente preme al completamento del programma. Una volta premuto il bottone, viene rivisualizzata la dashboard.

Visualizza Statistiche che permette di visualizzare tramite istogrammi il numero di volte che ogni programma è stato eseguito.

Upgrade che permette di sottoscrivere una iscrizione a `ROLE_USER_PRO`. Il pagamento è fittizio, si suppone che la selezione del piano implichi il pagamento stesso. Si segnalano che l'upgrade ha avuto successo.

Inserisci recensione che permette di inserire una nuova recensione della web app. L'utente inserisce una nuova recensione tramite una form. Il bottone e la form si trovano nella dashboard. L'invio della recensione, tramite il bottone Inserisci recensione, avviene tramite AJAX e la nuova recensione viene salvata nel database embedded e visualizzata immediatamente in un carosello (vedi più avanti) senza necessità di ricaricare la pagina.

Dashboard `ROLE_USER_PRO`:

La pagina Dashboard mostra un messaggio di benvenuto per l'utente (e.g. "Benvenuta Giovanna!") e offre le seguenti funzionalità abilitate premendo i pulsanti seguenti:

Visualizza Profilo che permette di accedere a una pagina con il profilo dell'utente (nome, cognome, data di nascita, ruolo).

Cambio Password che permette di cambiare la password associata all'utente tramite form.

Allenamento che permette di visualizzare in una pagina la lista dei programmi di allenamento *predefiniti* nella web app che vengono recuperati interrogando il servizio REST-like Programmi e dei programmi personalizzati inseriti manualmente e recuperati dal database della web app. Cliccando su ogni programma, viene visualizzata una scheda dettagliata del programma selezionato interrogando il servizio REST-like Programmi o il database della web app. Inoltre, c'è un bottone *Completato* che l'utente preme al completamento del programma. Una volta premuto il bottone, viene rivisualizzata la dashboard.

Inserisci Programma che permette di accedere ad una pagina dove l'utente può inserire un nuovo programma personalizzato. Il programma deve contenere dei componenti caratterizzati da *nome_esercizio*, *numero_serie*, *numero_ripetizioni* che sono inseriti dall'utente tramite una form. I dati relativi al nuovo programma vengono inseriti nel database della web app. Inoltre, viene segnalato che l'inserimento è andato a buon fine e viene interrogato il servizio REST-like Programmi per conoscere il numero di kcal associato al programma inserito.

Visualizza Statistiche che permette di visualizzare in una pagina, tramite istogrammi, il numero di volte che ogni programma è stato eseguito.

Inserisci recensione che permette di inserire una nuova recensione della web app. L'utente inserisce una nuova recensione tramite una form. Il bottone e la form si trovano nella dashboard. L'invio della recensione, tramite il bottone Inserisci recensione, avviene tramite AJAX e la nuova recensione viene

salvata nel database embedded e visualizzata immediatamente in un carosello (vedi più avanti) senza necessità di ricaricare la pagina.

Da tutte le pagine private diverse dalle Dashboard, deve essere possibile tornare alle Dashboard.

Attenzione! Una volta che gli utenti sono loggati, bisogna aggiungere la call to action logout e rimuovere le call to action Registrati e Inizia allenamento. Per esempio, si possono predisporre due barre di navigazione diverse che saranno incluse in modo esclusivo a seconda che l'utente sia loggato o meno.

Tutte le pagine pubbliche e private contengono un carosello di recensioni degli utenti. Le recensioni degli utenti vengono caricate dinamicamente tramite una chiamata AJAX/AJAJ al back end, che le recupera dal database embedded attraverso il servizio *Recensioni* interno alla web app e le restituisce alla pagina in formato dati; una volta ricevute, vengono visualizzate ~~nella home page~~ all'interno di un carosello Bootstrap, con ordine randomizzato e rotazione automatica ogni 30 secondi gestita da Bootstrap.

La web app usa una database H2 in modalità embedded. Per la gestione degli utenti, il database conterrà la tabella *Users* (che contiene ID, username, password, enabled) e la tabella *Authorities* (che contiene ID, username e il ruolo che può assumere l'utente) gestite (ma non create!) automaticamente da Spring Security. Ci sono poi altre tabelle. Ad esempio, i ROLE_USER_PRO useranno una tabella per contenere gli altri dati utente, una tabella per contenere i programmi di allenamento personalizzati, e una tabella per contenere il numero di volte in cui i programmi personalizzati sono stati usati. I ROLE_USER_BASIC e ROLE_USER_PROVA useranno soltanto la tabella per contenere gli altri dati utente. Queste tabelle saranno gestite da voi.

REST-like service Programmi

Si implementi un servizio REST like *Programmi* che permetta la gestione dei programmi di allenamento.

Il servizio ha tre endpoint che permettono di:

- 1) Ottenere la lista dei programmi di allenamento predefiniti. I nomi dei programmi predefiniti sono: *Full Body, Push/Pull/Legs, Cardio, Strength*
- 2) Ottenere, dato come parametro il programma di allenamento (e.g. Cardio), la composizione del programma. Ogni programma è composto da: *nome_esercizio, numero_serie, numero_ripetizioni, kcal*.
- 3) Ottenere, data un nuovo programma di allenamento, il numero di kcal associate al programma

Il servizio ha associato un database *programmiDB* che contiene la lista dei programmi e le loro informazioni dettagliate.

Suggerimento

Per la visualizzazione degli istogrammi si usi la libreria Chartjs che trovate qui:

<https://www.chartjs.org/>

Le condizioni di utilizzo sono a questo link:

<https://github.com/chartjs/Chart.js/blob/master/LICENSE.md>